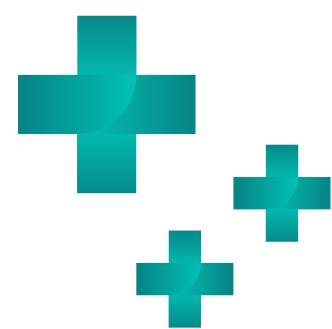
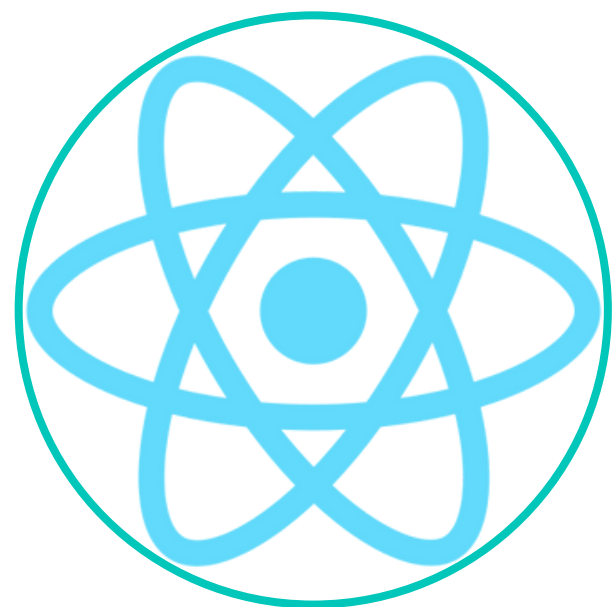


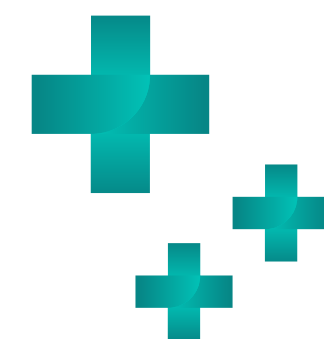


HEALTHSPHERE

Presented by:



**Di Giampaolo Angela
Pasquini Igor
Tutuianu Bogdan Andrei**



AGENDA

1. Introduction
2. Requirements
3. System Architecture
4. Complexity Estimation
5. Development Process
6. Implementation Details

INTRODUCTION

Healthsphere was born as a user-friendly online platform aiming to optimize and improve access to medical services.

Thanks to this system:

- + administrative overhead can be reduced
- + medical response times are improved
- + continuous care through digital channels becomes possible

REQUIREMENTS COLLECTION



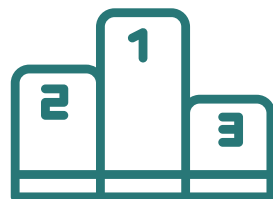
In order to implement the application, we adopted a user-centered approach.

In particular, we collected requirements in the form of **User Stories**, that allowed us to describe features from the end users' point of view. Each User Stories has the form:

As a [user], I want [feature], so that [goal]



3 users: patient, doctor, admin

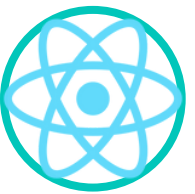


User Stories ordered by priority



HIGH PRIORITY USER STORIES

- **US1:** As a patient, I want to register on the platform and create a personal profile with my basic information so that I can access all the system's features.
- **US2:** As a patient, I want to update my profile with my medical history and insurance details so that my data remains current and useful for consultations.
- **US3:** As a patient, I want to search for doctors by specialty, location, availability, and language so that I can find a provider that best matches my needs.
- **US7:** As a patient, I want to upload my medical documents (lab results, prescriptions, images) so that doctors can review them ahead of appointments.
- **US8:** As a patient, I want to book an appointment with a selected doctor by choosing an available time slot so that I can secure a consultation at my convenience.
- **US13:** As a doctor, I want to create a detailed professional profile including my specialties, credentials, and experience so that patients can evaluate my qualifications.
- **US14:** As a doctor, I want to update my profile and upload supporting documents (certifications, licenses) so that my information remains verified and trustworthy.



HIGH PRIORITY USER STORIES

- **US15:** As a doctor, I want to set my working hours and update my calendar so that patients can book appointments only during my available times.
- **US16:** As a doctor, I want to receive notifications for new appointment requests so that I can promptly review and confirm or suggest alternative times.
- **US18:** As a doctor, I want to receive and reply to patient inquiries via chat or ticket so that I can provide timely advice even before consultations.
- **US20:** As a doctor, I want to generate and send digital prescriptions and medical reports to patients so that they can quickly access the necessary documentation.
- **US23:** As an admin, I want to view, modify, or deactivate user accounts (both patients and doctors) so that I can maintain the platform's integrity.
- **US24:** As an admin, I want to verify doctors' credentials and review their submitted documentation so that only qualified professionals are active on the platform.
- **US25:** As an admin, I want to monitor system performance and usage metrics so that I can quickly detect and resolve issues.
- **US26:** As an admin, I want to configure and update security policies and access permissions so that patient and doctor data remains protected.



MEDIUM PRIORITY USER STORIES

- **US4:** As a patient, I want to apply filters (e.g., rating, distance, fee range) and sort search results so that I can quickly narrow down the best options.
- **US5:** As a patient, I want to initiate a chat or submit a support ticket to a doctor so that I can ask pre-consultation questions.
- **US9:** As a patient, I want to view, reschedule, or cancel my appointments so that I can maintain flexibility in my schedule.
- **US10:** As a patient, I want to receive and view digital prescriptions and follow-up instructions so that I have ready access to treatment details.
- **US17:** As a doctor, I want to view and manage my upcoming appointments in a calendar view so that I can effectively organize my schedule.
- **US19:** As a doctor, I want to view and annotate uploaded patient documents so that I can prepare in advance for each consultation.

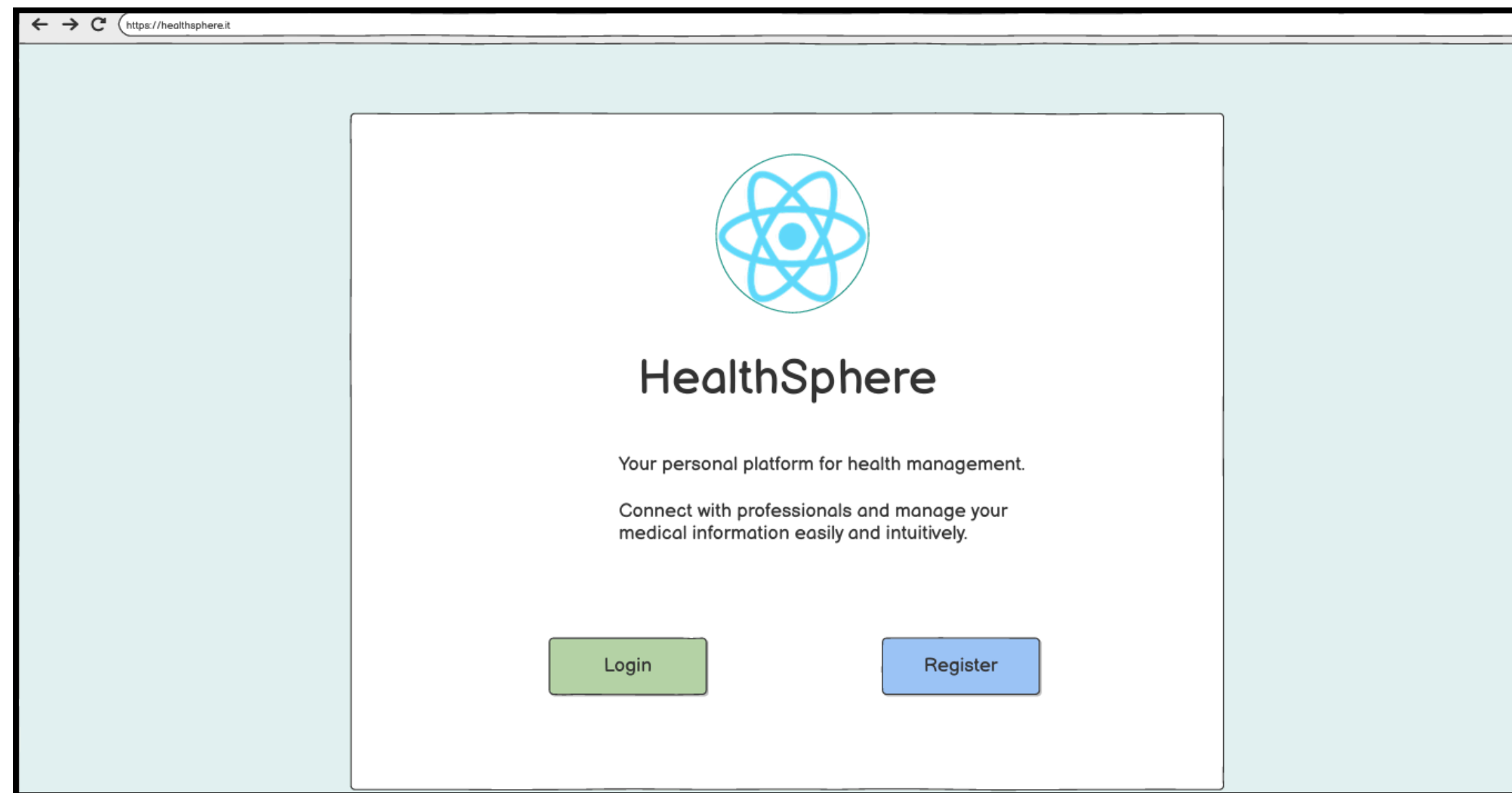
LOW PRIORITY USER STORIES

- **US6:** As a patient, I want to view the history of my chats or tickets with doctors so that I can refer back to previous advice.
- **US12:** As a patient, I want to leave a review and rate a doctor after an appointment so that other patients can make informed decisions.
- **US22:** As a doctor, I want to see the reviews and ratings left by my patients so that I can understand their feedback.

WIREFRAMES

We used Balsamiq in order to realize low-fidelity mockup of user interfaces.

In this way we simulated the behavior of the applications and the possible users' interactions.
Here there are some examples:

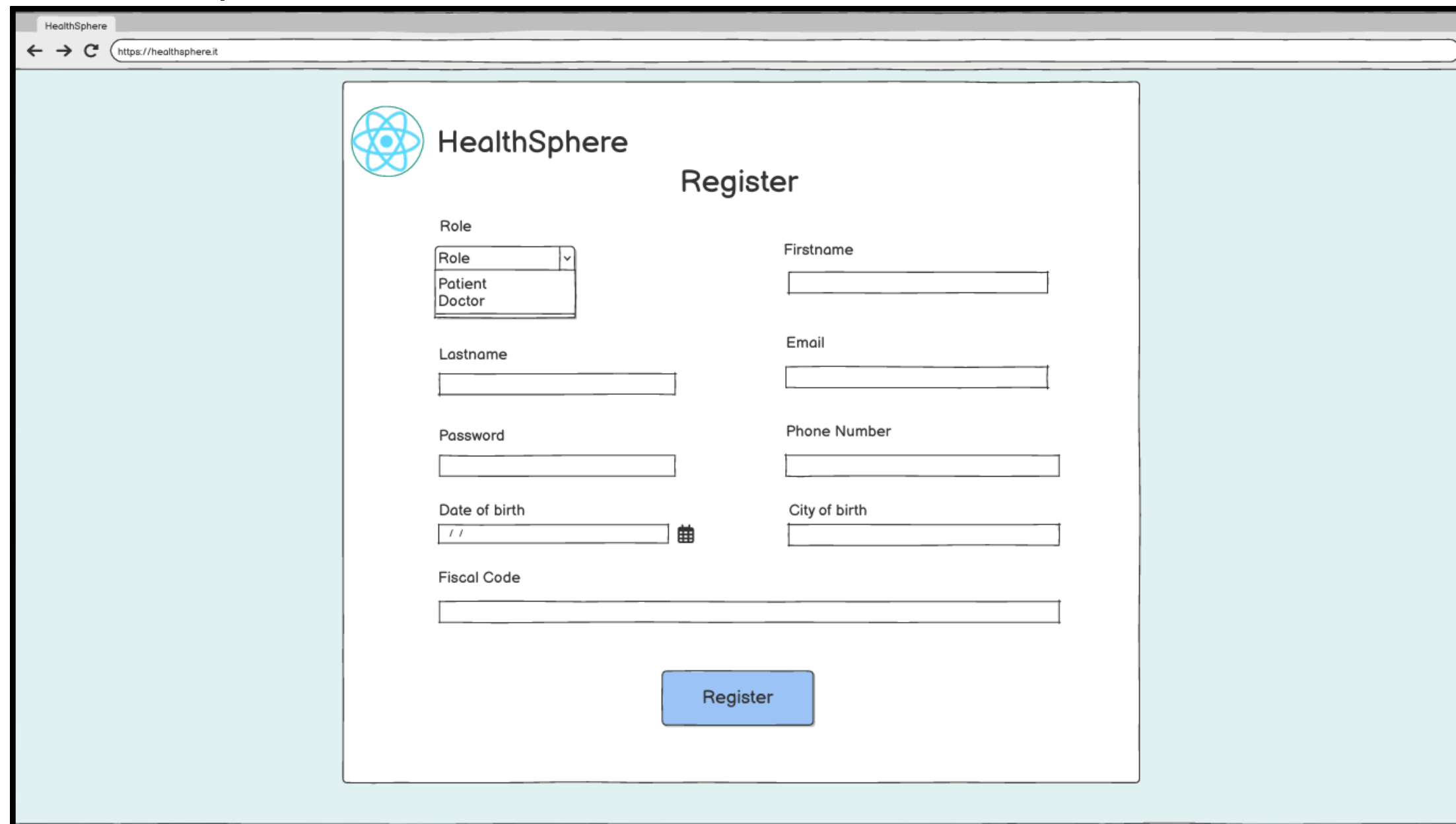


WIREFRAMES

We used Balsamiq in order to realize low-fidelity mockup of user interfaces.

In this way we simulated the behavior of the applications and the possible users' interactions.

Here there are some examples:



The image shows a wireframe of a web browser displaying a registration form for HealthSphere. The browser's address bar shows the URL <https://healthsphere.it>. The form is titled "Register" and includes the HealthSphere logo. The form fields are arranged in two columns:

- Left Column:**
 - Role: A dropdown menu with "Patient" and "Doctor" as options.
 - Lastname: A text input field.
 - Password: A text input field.
 - Date of birth: A text input field with a calendar icon.
 - Fiscal Code: A text input field.
- Right Column:**
 - Firstname: A text input field.
 - Email: A text input field.
 - Phone Number: A text input field.
 - City of birth: A text input field.

A "Register" button is located at the bottom center of the form.

SYSTEM ARCHITECTURE

There are three main users, each one can access the application from a laptop or a mobile device.



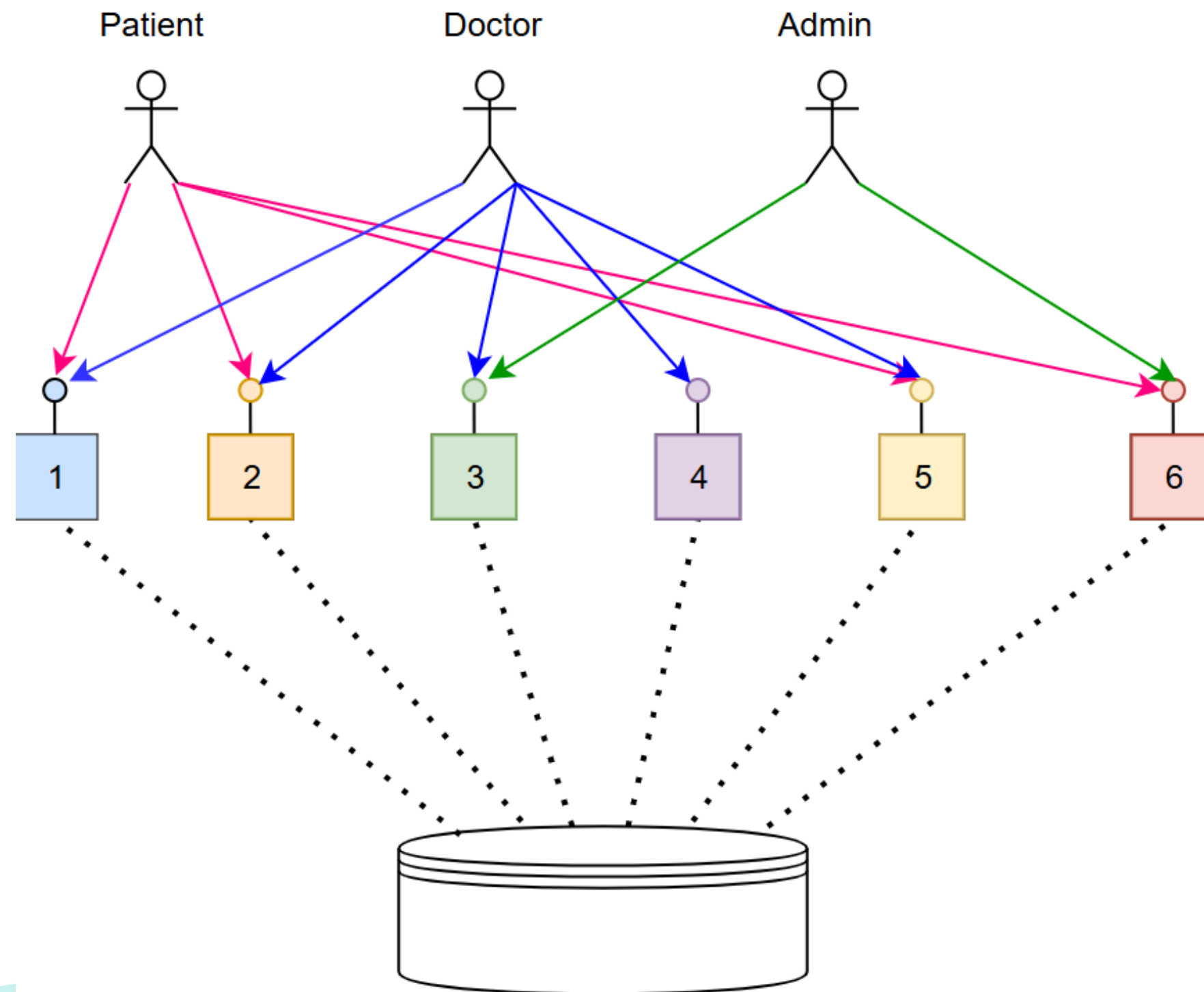
SYSTEM ARCHITECTURE

We implemented a microservice architecture.
We built the following microservices:

1. **Appointment Service** → allows the implementation of the appointment mechanism between patients and doctors
2. **Communication Service** → allows the communication between patients and doctors through a chat system
3. **Doctor Service** → allows the doctors' management, implementing functionality for the uploads of certificates and doctors' career management
4. **Document Service** → allows the management of uploaded and exchanged documents
5. **Review Service** → implements the review system, according to which patients can leave review to doctors
6. **User Service** → allows patients management

SYSTEM ARCHITECTURE

A schema for this system can be:

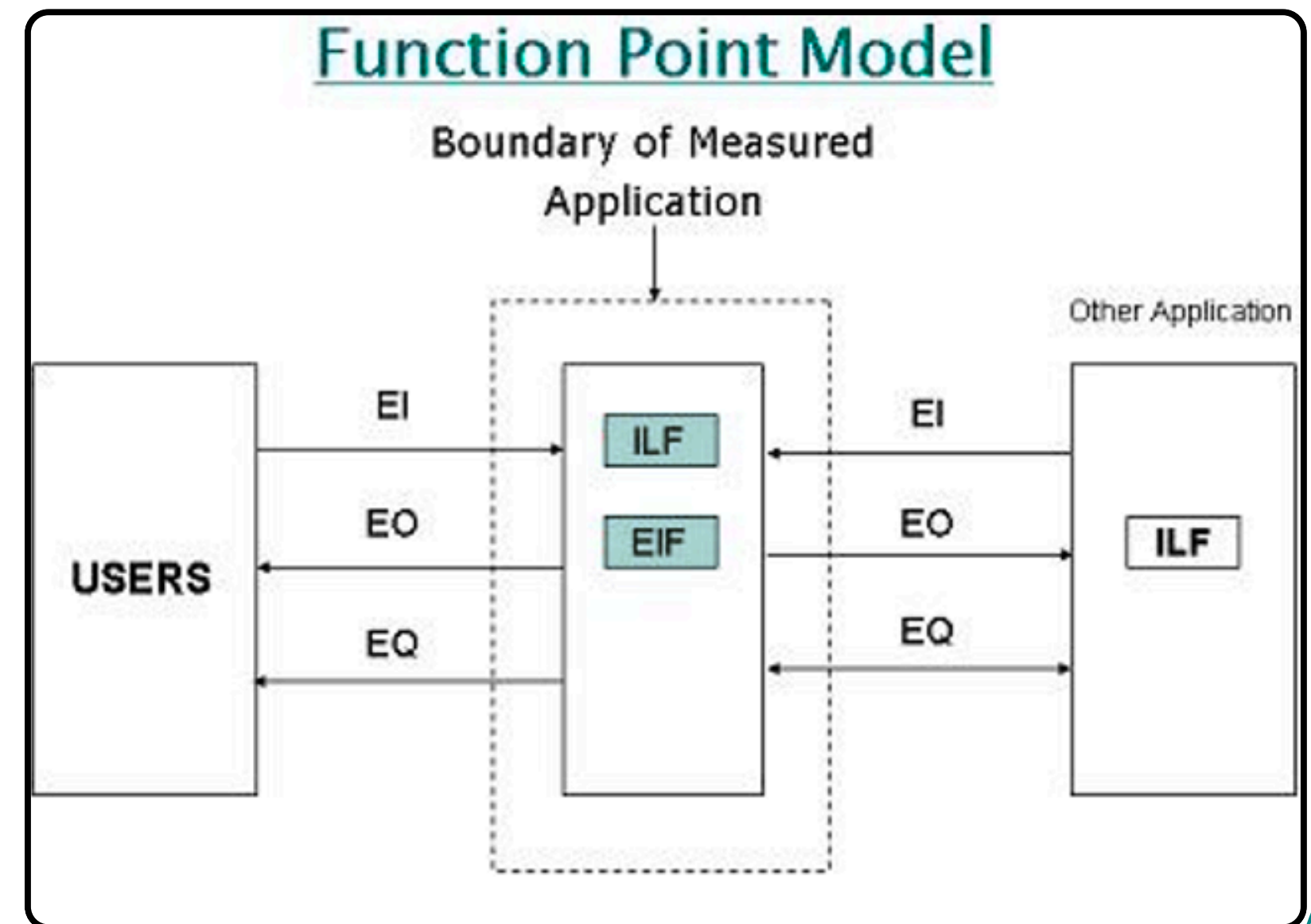


COMPLEXITY ESTIMATION

We estimated the size and the complexity of our project through **Function Points**, a method that allows to measure the functionality delivered by a software system based on:

- user inputs
- outputs
- data files
- interactions

In order to estimate the effort, the time and the cost required to develop this software, we adopted **COCOMO II** (Constructive Cost Model II).





COMPLEXITY ESTIMATION

ILF	110 FP
EIF	0 FP
EI	115 FP
EQ	47 FP
EO	28 FP



TOT: 330 UFP

COCOMO II

Initial assumption: the project si composed by:

- 60% backend Node.js
- 30% frontend React
- 10% database SQL

1. From function point to SLOC

On average: $(0.60 \times 47) + (0.30 \times 46) + (0.10 \times 13) = 43.3 \text{ LOC/FP}$

Total: $330 \times 43.3 = 14.289 \rightarrow 14.3 \text{ KSLOC}$

2. Scale Drivers

- PREC (precedentedness) = 3.72
- FLEX (Development Flexibility) = 2.03
- RESL (Risk Resolution) = 3.29
- TEAM (Team Cohesion) = 2.83
- PMAT (Process Maturity) = 6.24
- Total: 18.11

$$E = 0.91 + 0.01 \times 18.11 = 1.09$$

COCOMO II

3. Effort Multipliers

EAF (Effort Adjustment Factor) = 1.85

4. COCOMOII Estimation

We used the Early Design approach

$$Effort = A \times (Size^E) \times EAF$$

- $A = 2.94$
- $Size = 14.3$ KSLOC
- $E = 1.09$
- $EAF = 1.85$

In total: 92.3 person-months.

From the duration point of view, TDEV = 14.2 months

Estimated team size: $92.3/14.2 = 6.5$ developers

COCOMO II

Real Estimation

We have to consider that:

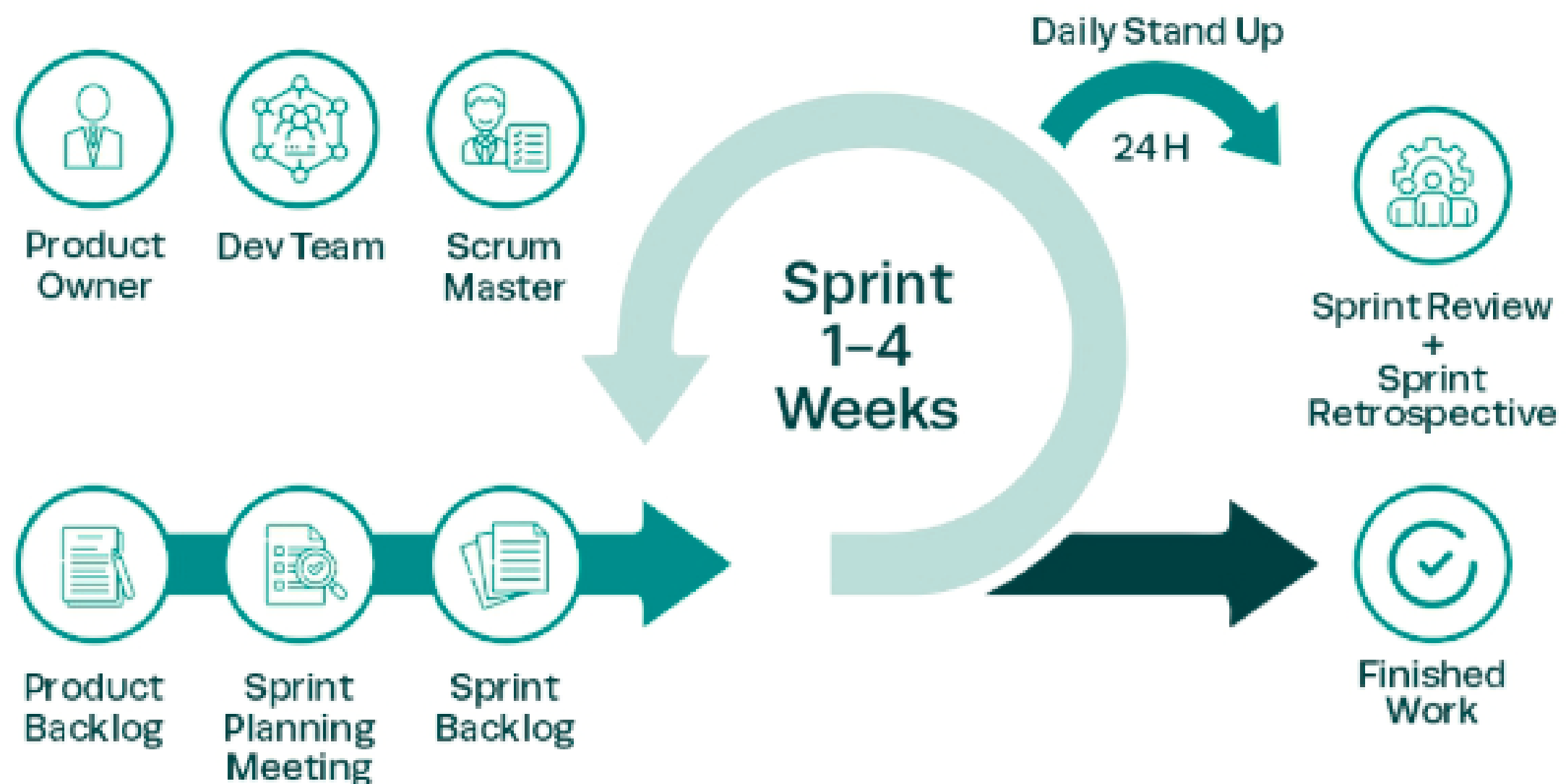
- Team: 3 students
- Maximum duration: 5 months
- Total Effort: $3 \times 5 = 15/16$ person-months

The real productivity is:

$$14.289/16 = 893 \text{ SLOC/person-months}$$

SCRUM

We adopted a SCRUM approach in which each Sprint lasts 2 weeks



SCRUM

Sprint 0: Project setup → environment, CI/CD, DB schema, API Gateway, docs

Sprint 1: Core authentication → patient registration (US1), doctor profile (US12), authentication service, login/register pages

Sprint 2: Profile management → patient update (US2), doctor update & upload (US13), admin management (US21), profile pages, file upload

Sprint 3: Doctor search → search doctors (US3), filters/sort (US4), view reviews (US20), admin credential verification (US22), search UI

Sprint 4: Appointment system core → book appointment (US8), doctor availability/calendar (US14), backend service, calendar integration

SCRUM

Sprint 5 : Appointment lifecycle → manage appointments (US9), notifications (US15), doctor tools (US16), frontend pages

Sprint 6: Communication system → patient chat (US5), chat history (US6), doctor-patient messaging (US17), backend + real-time UI

Sprint 7: Document management → upload documents (US7), annotate/view (US18), digital prescriptions (US19, US10), frontend module

Sprint 8: Reviews & admin tools → leave reviews (US11), system monitoring (US23), security policies (US24), review UI, admin dashboard

Sprint 9: Integration & testing → end-to-end tests, performance, security fixes, UI/UX (no new US)

BACKEND



Each microservice runs on a **Docker container**. In particular we have :

- a *docker-file* for each microservice → it defines how to build a docker image and it specifies the base image, the dependencies, environment settings, and commands to run the microservice
- a *docker-compose* file → it is used to manage multi-container applications



In order to store and retrieve data, we used **PostgreSQL**, a powerful open-source relational database that ensures reliable data storage, complex queries, and scalability.

We also implemented the **API Gateway**, that acts as the single entry point for client requests, routing them to the appropriate microservices, enforcing security, rate limiting, and centralized error handling. Provides health checks and service discovery for the microservices ecosystem."

FRONTEND

The frontend is built with **React**, delivering a dynamic, responsive, and interactive user interface.

It allows patients and doctors to interact seamlessly with the platform, managing profiles, appointments, chats, and medical documents.

- Responsive design and styling using Bootstrap and React-Bootstrap for consistent layouts across desktop, tablet, and mobile devices.
- Component-based architecture with reusable UI components (buttons, forms, modals, cards).
- State management using React Context or Redux to handle authentication, profiles, and chat sessions.
- Form validation with Formik and Yup to ensure data integrity before submission.

The background is white with several teal-colored abstract elements. In the top left, there is a series of concentric, overlapping curved lines that form a fan-like shape. In the top right, there is a large, solid teal circle. In the bottom left, there is a large, solid teal shape with a wavy, organic edge. In the bottom right, there is another series of concentric, overlapping curved lines, similar to the one in the top left. The text "THANK YOU FOR THE ATTENTION" is centered in the middle of the image in a bold, teal, sans-serif font.

**THANK YOU FOR THE
ATTENTION**